

ITA0608 - Machine Learning

List of Lab Exercises

3. Demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and applying this knowledge to classify a new sample.

Aim:

To implement and demonstrate the ID3 Decision Tree Algorithm using a training dataset, construct a decision tree based on information gain, and classify a new sample.

Algorithm:

1. Import the required libraries.
2. Load the Play Tennis dataset from a CSV file.
3. Preprocess the dataset if necessary.
4. Train the Decision Tree model using the ID3 algorithm.
5. Construct the decision tree based on the attribute with the highest information gain.
6. Input a new sample for testing.
7. Predict the class label of the new sample.
8. Display the prediction result.

Dataset 1:

Program Code:

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.tree import DecisionTreeClassifier, plot_tree
data = pd.read_excel("play_tennis_dataset.xlsx")

print("\nOriginalDataset\n") print(data)

encoded_data = data.copy()
```

```

encoding_dictionary = {}

for column in encoded_data.columns:

    unique_values = encoded_data[column].unique()

    value_map = {value:index for index,value in enumerate(unique_values)} encoding_dictionary[column] = value_map

    encoded_data[column]=encoded_data[column].map(value_map) print("\nEncoded Dataset\n")

print(encoded_data)

features = encoded_data.drop("PlayTennis", axis=1) target =

encoded_data["PlayTennis"]

classifier = DecisionTreeClassifier(

    criterion="entropy", random_state=10

)

classifier.fit(features, target)

print("\nModel Training Completed Successfully.")

plt.figure(figsize=(13,8)) plot_tree(

    classifier, feature_names=features.columns,

    class_names=["No","Yes"], filled=True,

```

```

        rounded=True, fontsize=10

    )

plt.title("ID3 Decision Tree") plt.show()

sample = {

    "Outlook":"Sunny", "Temperature":"Cool",

    "Humidity":"High",

    "Wind":"Weak"

}

sample_df=pd.DataFrame([sample]) for column in

sample_df.columns:

    sample_df[column]=sample_df[column].map(encoding_dictionary[column]) prediction =

classifier.predict(sample_df)

reverse_output = {v:k for k,v in encoding_dictionary["PlayTennis"].items()} print("\n===== Prediction

=====")

print("Input Sample") print(sample)

print(" -----")

print("Predicted Class :", reverse_output[prediction[0]])

print("=====")

```

Output:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier, plot_tree

# Read dataset directly as 'play_tennis_dataset.xlsx' is available
data = pd.read_excel("play_tennis_dataset.xlsx")

print("\nOriginal Dataset\n")
print(data)

# Convert categorical values into numeric values
encoded_data = data.copy()

encoding_dictionary = {}

for column in encoded_data.columns:
    unique_values = encoded_data[column].unique()
    value_map = {value:index for index,value in enumerate(unique_values)}
    encoding_dictionary[column] = value_map
    encoded_data[column] = encoded_data[column].map(value_map)

print("\nEncoded Dataset\n")
print(encoded_data)

# Separate features and target
features = encoded_data.drop(["PlayTennis", axis=1])
target = encoded_data["PlayTennis"]

# Build Decision Tree using Entropy (ID3)
classifier = DecisionTreeClassifier(
    criterion="entropy",
    random_state=10
)

classifier.fit(features, target)

print("\nModel Training Completed Successfully.")
```



```
plt.figure(figsize=(13,8))

plot_tree(
    classifier,
    feature_names=features.columns,
    class_names=["No", "Yes"],
    filled=True,
    rounded=True,
    fontsize=10
)

plt.title("ID3 Decision Tree")
plt.show()

# -----
# Test New Sample
# -----

sample = {
    "Outlook": "Sunny",
    "Temperature": "Cool",
    "Humidity": "High",
    "Wind": "Weak"
}

sample_df = pd.DataFrame([sample])

for column in sample_df.columns:
    sample_df[column] = sample_df[column].map(encoding_dictionary[column])

prediction = classifier.predict(sample_df)

reverse_output = {v:k for k,v in encoding_dictionary["PlayTennis"].items()}

print("\n===== Prediction =====")
print("Input Sample")
print(sample)
print("-----")
print("Predicted Class :", reverse_output[prediction[0]])
print("=====")
```

```
print("=====")
```

...

Original Dataset

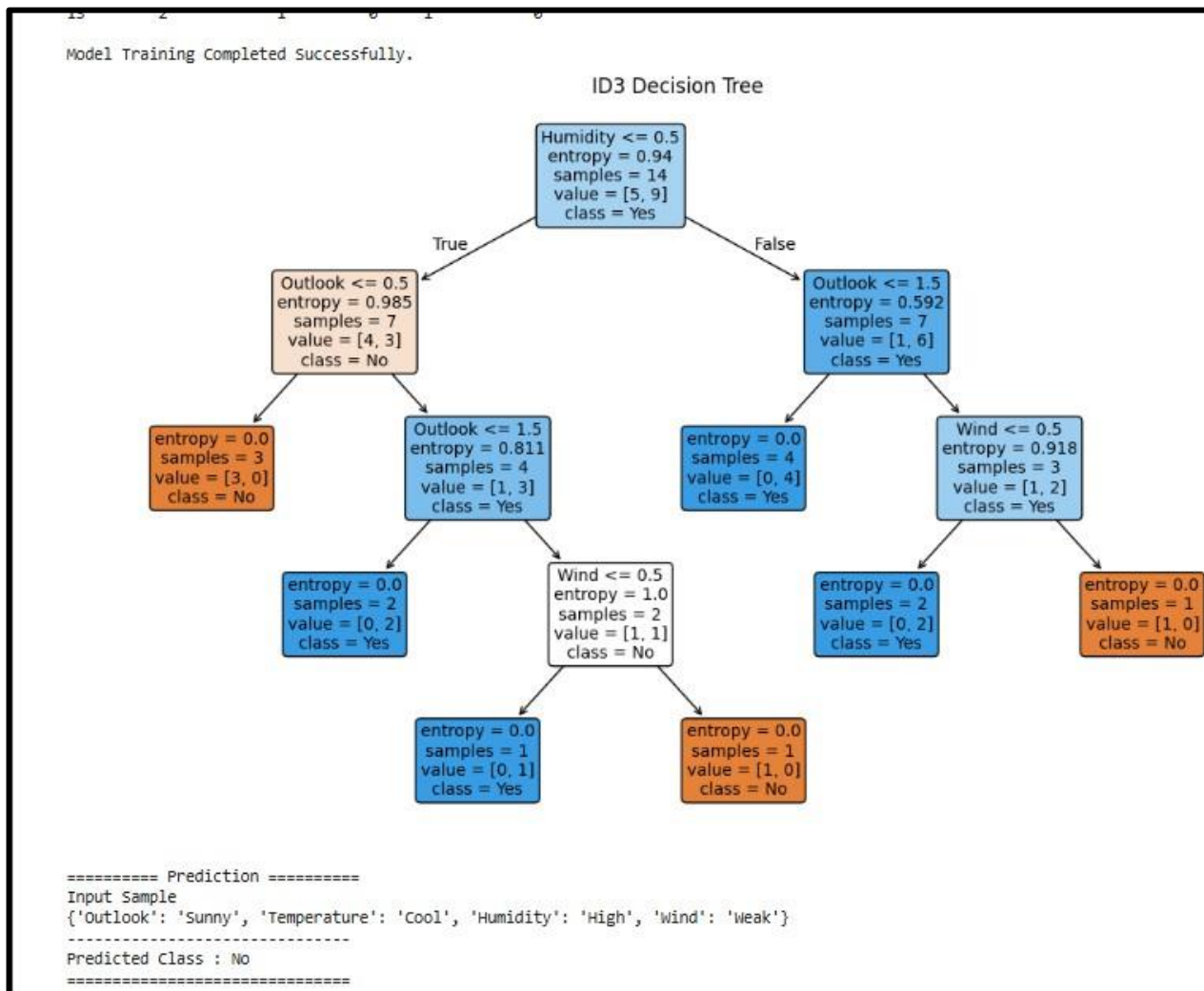
	Outlook	Temperature	Humidity	Wind	PlayTennis
0	Sunny	Hot	High	Weak	No
1	Sunny	Hot	High	Strong	No
2	Overcast	Hot	High	Weak	Yes
3	Rain	Mild	High	Weak	Yes
4	Rain	Cool	Normal	Weak	Yes
5	Rain	Cool	Normal	Strong	No
6	Overcast	Cool	Normal	Strong	Yes
7	Sunny	Mild	High	Weak	No
8	Sunny	Cool	Normal	Weak	Yes
9	Rain	Mild	Normal	Weak	Yes
10	Sunny	Mild	Normal	Strong	Yes
11	Overcast	Mild	High	Strong	Yes
12	Overcast	Hot	Normal	Weak	Yes
13	Rain	Mild	High	Strong	No

Encoded Dataset

	Outlook	Temperature	Humidity	Wind	PlayTennis
0	0	0	0	0	0
1	0	0	0	1	0
2	1	0	0	0	1
3	2	1	0	0	1
4	2	2	1	0	1
5	2	2	1	1	0
6	1	2	1	1	1
7	0	1	0	0	0
8	0	2	1	0	1
9	2	1	1	0	1
10	0	1	1	1	1
11	1	1	0	1	1
12	1	0	1	0	1
13	2	1	0	1	0

Model Training Completed Successfully.

ID3 Decision Tree



Dataset 2:

Program Code:

```
import pandas as pd
import numpy as np
from sklearn.tree import DecisionTreeClassifier, export_text
```

```
data = [
    ['High', 'Good', 'Yes', 'Good', 'Yes'],
    ['High', 'Excellent', 'Yes', 'Excellent', 'Yes'],
    ['Medium', 'Good', 'Yes', 'Good', 'Yes'],
    ['Medium', 'Average', 'No', 'Average', 'No'],
    ['Low', 'Poor', 'No', 'Poor', 'No'],
    ['High', 'Good', 'No', 'Good', 'Yes'],
    ['Low', 'Average', 'No', 'Average', 'No'],
    ['Medium', 'Good', 'Yes', 'Excellent', 'Yes'],
    ['High', 'Excellent', 'Yes', 'Good', 'Yes'],
    ['High', 'Good', 'Yes', 'Good', 'Yes'],
    ['Low', 'Poor', 'Yes', 'Average', 'No'],
    ['Low', 'Average', 'No', 'Average', 'No']
]
```

```

columns = [
    'CGPA',
    'Communication',
    'Internship',
    'Programming',
    'Placement'
]
df = pd.DataFrame(data, columns=columns)

print("Student Placement Dataset:")
print(df)

X = df[['CGPA', 'Communication', 'Internship', 'Programming']]
y = df['Placement']

X_encoded = pd.get_dummies(X)

print("\nEncoded Training Data:")
print(X_encoded)

model = DecisionTreeClassifier(
    criterion='entropy',
    random_state=42
)

model.fit(X_encoded, y)

new_sample = pd.DataFrame([
    ['High', 'Good', 'Yes', 'Excellent']
], columns=[
    'CGPA',
    'Communication',
    'Internship',
    'Programming'
])

new_sample_encoded = pd.get_dummies(new_sample)

new_sample_encoded = new_sample_encoded.reindex(
    columns=X_encoded.columns,
    fill_value=0
)

prediction = model.predict(new_sample_encoded)

```

```

print("\n-----")
print("Decision Tree Prediction")
print("-----")

print("\nNew Sample:")
print(new_sample)

print("\nPredicted Result:")
print(prediction[0])

print("\nDecision Tree Rules:")
print(
    export_text(
        model,
        feature_names=list(X_encoded.columns)
    )
)

```

Output:

```

*** Student Placement Dataset:
      CGPA Communication Internship Programming Placement
0      High      Good      Yes      Good      Yes
1      High      Excellent      Yes      Excellent      Yes
2      Medium      Good      Yes      Good      Yes
3      Medium      Average      No      Average      No
4      Low      Poor      No      Poor      No
5      High      Good      No      Good      Yes
6      Low      Average      No      Average      No
7      Medium      Good      Yes      Excellent      Yes
8      High      Excellent      Yes      Good      Yes
9      High      Good      Yes      Good      Yes
10     Low      Poor      Yes      Average      No
11     Low      Average      No      Average      No

Encoded Training Data:
      CGPA_High CGPA_Low CGPA_Medium Communication_Average \
0      True      False      False      False      False
1      True      False      False      False      False
2      False     False      True      False      False
3      False     False      True      True      True
4      False     True      False      False      False
5      True      False      False      False      False
6      False     True      False      True      True
7      False     False      True      False      False
8      True      False      False      False      False
9      True      False      False      False      False
10     False     True      False      False      False
11     False     True      False      True      True

```

```

***
Communication_Excellent Communication_Good Communication_Poor \
0 False True False
1 True False False
2 False True False
3 False False False
4 False False True
5 False True False
6 False False False
7 False True False
8 True False False
9 False True False
10 False False True
11 False False False

Internship_No Internship_Yes Programming_Average Programming_Excellent \
0 False True False False
1 False True False True
2 False True False False
3 True False True False
4 True False False False
5 True False False False
6 True False True False
7 False True False True
8 False True False False
9 False True False False
10 False True True False
11 True False True False

```

```

2 True False
3 False False
*** 4 False True
5 True False
6 False False
7 False False
8 True False
9 True False
10 False False
11 False False

```

Decision Tree Prediction

New Sample:
CGPA Communication Internship Programming
0 High Good Yes Excellent

Predicted Result:
Yes

Decision Tree Rules:
|--- CGPA_Low <= 0.50
| |--- Programming_Average <= 0.50
| | |--- class: Yes
| |--- Programming_Average > 0.50
| | |--- class: No
|--- CGPA_Low > 0.50
| |--- class: No

Dataset 3:

Program Code:

```
import pandas as pd
import numpy as np
from sklearn.tree import DecisionTreeClassifier, export_text

data = [
    ['High', 'Good', 'Permanent', 'Yes', 'Yes'],
    ['High', 'Good', 'Permanent', 'No', 'Yes'],
    ['Medium', 'Good', 'Permanent', 'Yes', 'Yes'],
    ['Low', 'Poor', 'Temporary', 'No', 'No'],
    ['Medium', 'Average', 'Permanent', 'No', 'Yes'],
    ['High', 'Average', 'Temporary', 'Yes', 'No'],
    ['Low', 'Poor', 'Temporary', 'Yes', 'No'],
    ['Medium', 'Good', 'Permanent', 'Yes', 'Yes'],
    ['High', 'Good', 'Permanent', 'Yes', 'Yes'],
    ['Low', 'Average', 'Temporary', 'No', 'No']
]

columns = [
    'Income',
    'Credit Score',
    'Employment',
    'Property',
    'Loan Approved'
]
df = pd.DataFrame(data, columns=columns)

print("Loan Approval Dataset:")
print(df)

X = df[['Income', 'Credit Score', 'Employment', 'Property']]
y = df['Loan Approved']

X_encoded = pd.get_dummies(X)

print("\nEncoded Training Data:")
print(X_encoded)

model = DecisionTreeClassifier(
    criterion='entropy',
    random_state=42
)

model.fit(X_encoded, y)
```

```
print("\nModel Training Completed Successfully.")
```

```
new_sample = pd.DataFrame([
    ['Medium', 'Good', 'Permanent', 'Yes']
], columns=[
    'Income',
    'Credit Score',
    'Employment',
    'Property'
])
```

```
new_sample_encoded = pd.get_dummies(new_sample)
```

```
new_sample_encoded = new_sample_encoded.reindex(
    columns=X_encoded.columns,
    fill_value=0
)
```

```
prediction = model.predict(new_sample_encoded)
```

```
print("\n-----")
print("Decision Tree Prediction")
print("-----")
```

```
print("\nNew Sample:")
print(new_sample)
```

```
print("\nPredicted Result:")
print(prediction[0])
```

```
print("\nDecision Tree Rules:")
```

```
print(
    export_text(
        model,
        feature_names=list(X_encoded.columns)
    )
)
```

Output:

```
... Loan Approval Dataset:
  Income Credit Score Employment Property Loan Approved
0   High      Good  Permanent      Yes      Yes
1   High      Good  Permanent      No       Yes
2  Medium      Good  Permanent      Yes      Yes
3    Low      Poor  Temporary      No       No
4  Medium  Average  Permanent      No       Yes
5   High  Average  Temporary      Yes      No
6    Low      Poor  Temporary      Yes      No
7  Medium      Good  Permanent      Yes      Yes
8   High      Good  Permanent      Yes      Yes
9    Low  Average  Temporary      No       No

Encoded Training Data:
  Income_High Income_Low Income_Medium Credit Score_Average \
0         True      False      False      False      False
1         True      False      False      False      False
2        False      False      True       True       False
3        False      True       False      False      False
4        False      False      True       True       True
5         True      False      False      False      True
6        False      True       False      False      False
7        False      False      True       True       False
8         True      False      False      False      False
9        False      True       False      False      True
```

```
...
  Credit Score_Good Credit Score_Poor Employment_Permanent \
0             True      False      True
1             True      False      True
2             True      False      True
3            False      True       False
4            False      False      True
5            False      False      False
6            False      True       False
7             True      False      True
8             True      False      True
9            False      False      False

  Employment_Temporary Property_No Property_Yes
0             False      False      True
1             False      True      False
2             False      False      True
3              True      True      False
4             False      True      False
5              True      False      True
6              True      False      True
7             False      False      True
8             False      False      True
9              True      True      False
```

Model Training Completed Successfully.

```
Model Training Completed Successfully.
```

```
-----  
Decision Tree Prediction  
-----
```

```
New Sample:
```

```
Income Credit Score Employment Property  
0 Medium          Good Permanent      Yes
```

```
Predicted Result:
```

```
Yes
```

```
Decision Tree Rules:
```

```
|--- Employment_Permanent <= 0.50  
|   |--- class: No  
|--- Employment_Permanent > 0.50  
|   |--- class: Yes
```

Dataset 4:

Program Code:

```
import pandas as pd  
import numpy as np  
from sklearn.tree import DecisionTreeClassifier, export_text
```

```
data = [  
    ['Yes', 'Yes', 'Yes', 'Yes', 'Positive'],  
    ['Yes', 'Yes', 'No', 'Yes', 'Positive'],  
    ['No', 'Yes', 'Yes', 'No', 'Negative'],  
    ['Yes', 'Yes', 'Yes', 'No', 'Positive'],  
    ['No', 'No', 'Yes', 'Yes', 'Negative'],  
    ['Yes', 'Yes', 'No', 'No', 'Positive'],  
    ['Yes', 'No', 'Yes', 'Yes', 'Positive'],  
    ['No', 'No', 'No', 'No', 'Negative']  
]
```

```
columns = [  
    'Fever',  
    'Cough',  
    'Headache',  
    'Body Pain',  
    'Disease'  
]
```

```
df = pd.DataFrame(data, columns=columns)
```

```
print("Disease Diagnosis Dataset:")  
print(df)
```

```
X = df[['Fever', 'Cough', 'Headache', 'Body Pain']]  
y = df['Disease']
```

```

X_encoded = pd.get_dummies(X)

print("\nEncoded Training Data:")
print(X_encoded)

model = DecisionTreeClassifier(
    criterion='entropy',
    random_state=42
)

model.fit(X_encoded, y)

print("\nModel Training Completed Successfully.")

new_sample = pd.DataFrame([
    ['Yes', 'Yes', 'No', 'Yes']
], columns=[
    'Fever',
    'Cough',
    'Headache',
    'Body Pain'
])

new_sample_encoded = pd.get_dummies(new_sample)

new_sample_encoded = new_sample_encoded.reindex(
    columns=X_encoded.columns,
    fill_value=0
)

prediction = model.predict(new_sample_encoded)

print("\n-----")
print("Decision Tree Prediction")
print("-----")

print("\nNew Sample:")
print(new_sample)

print("\nPredicted Result:")
print(prediction[0])

print("\nDecision Tree Rules:")

print(

```

```

export_text(
    model,
    feature_names=list(X_encoded.columns)
)

```

Output:

```

Disease Diagnosis Dataset:
***
  Fever Cough Headache Body Pain  Disease
0   Yes   Yes    Yes    Yes  Positive
1   Yes   Yes    No    Yes  Positive
2   No    Yes    Yes    No   Negative
3   Yes   Yes    Yes    No   Positive
4   No    No    Yes    Yes  Negative
5   Yes   Yes    No    No   Positive
6   Yes   No    Yes    Yes  Positive
7   No    No    No    No   Negative

Encoded Training Data:
  Fever_No Fever_Yes Cough_No Cough_Yes Headache_No Headache_Yes \
0   False    True   False    True    False    True
1   False    True   False    True    True    False
2   True     False  False    True    False    True
3   False    True   False    True    False    True
4   True     False  True     False   False    True
5   False    True   False    True    True    False
6   False    True   True     False   False    True
7   True     False  True     False   True     False

```

```

  Body Pain_No Body Pain_Yes
***
0   False    True
1   False    True
2   True     False
3   True     False
4   False    True
5   True     False
6   False    True
7   True     False

Model Training Completed Successfully.

-----
Decision Tree Prediction
-----

New Sample:
  Fever Cough Headache Body Pain
0   Yes   Yes    No    Yes

Predicted Result:
Positive

Decision Tree Rules:
|--- Fever_No <= 0.50
|   |--- class: Positive
|--- Fever_No > 0.50
|   |--- class: Negative

```

Dataset 5:

Program Code:

```
import pandas as pd
import numpy as np
from sklearn.tree import DecisionTreeClassifier, export_text

data = [
    ['High', 'Excellent', 'Yes', 'Yes', 'Yes'],
    ['High', 'Good', 'Yes', 'Yes', 'Yes'],
    ['Medium', 'Good', 'No', 'Yes', 'Yes'],
    ['Low', 'Average', 'No', 'No', 'No'],
    ['Medium', 'Excellent', 'Yes', 'Yes', 'Yes'],
    ['Low', 'Poor', 'No', 'No', 'No'],
    ['High', 'Good', 'Yes', 'No', 'Yes'],
    ['Medium', 'Average', 'No', 'Yes', 'No'],
    ['High', 'Excellent', 'Yes', 'Yes', 'Yes'],
    ['Low', 'Poor', 'No', 'Yes', 'No']
]

columns = [
    'Experience',
    'Performance',
    'Leadership',
    'Training',
    'Promotion'
]
df = pd.DataFrame(data, columns=columns)

print("Employee Promotion Dataset:")
print(df)

X = df[
    ['Experience', 'Performance', 'Leadership', 'Training']
]

y = df['Promotion']

X_encoded = pd.get_dummies(X)

print("\nEncoded Training Data:")
print(X_encoded)

model = DecisionTreeClassifier(
    criterion='entropy',
    random_state=42
)
```

```

model.fit(X_encoded, y)

print("\nModel Training Completed Successfully.")

new_sample = pd.DataFrame([
    ['High', 'Excellent', 'Yes', 'Yes']
], columns=[
    'Experience',
    'Performance',
    'Leadership',
    'Training'
])

new_sample_encoded = pd.get_dummies(new_sample)

new_sample_encoded = new_sample_encoded.reindex(
    columns=X_encoded.columns,
    fill_value=0
)

prediction = model.predict(new_sample_encoded)

print("\n-----")
print("Decision Tree Prediction")
print("-----")

print("\nNew Sample:")
print(new_sample)

print("\nPredicted Result:")
print(prediction[0])

print("\nDecision Tree Rules:")

print(
    export_text(
        model,
        feature_names=list(X_encoded.columns)
    )
)

```

Output:

*** Employee Promotion Dataset:

	Experience	Performance	Leadership	Training	Promotion
0	High	Excellent	Yes	Yes	Yes
1	High	Good	Yes	Yes	Yes
2	Medium	Good	No	Yes	Yes
3	Low	Average	No	No	No
4	Medium	Excellent	Yes	Yes	Yes
5	Low	Poor	No	No	No
6	High	Good	Yes	No	Yes
7	Medium	Average	No	Yes	No
8	High	Excellent	Yes	Yes	Yes
9	Low	Poor	No	Yes	No

Encoded Training Data:

	Experience_High	Experience_Low	Experience_Medium	Performance_Average	\
0	True	False	False	False	
1	True	False	False	False	
2	False	False	True	False	
3	False	True	False	True	
4	False	False	True	False	
5	False	True	False	False	
6	True	False	False	False	
7	False	False	True	True	
8	True	False	False	False	
9	False	True	False	False	

	Performance_Excellent	Performance_Good	Performance_Poor	Leadership_No	\
0	True	False	False	False	
1	False	True	False	False	
2	False	True	False	True	
3	False	False	False	True	
4	True	False	False	False	
5	False	False	True	True	
6	False	True	False	False	
7	False	False	False	True	
8	True	False	False	False	
9	False	False	True	True	

	Leadership_Yes	Training_No	Training_Yes
0	True	False	True
1	True	False	True
2	False	False	True
3	False	True	False
4	True	False	True
5	False	True	False
6	True	True	False
7	False	False	True
8	True	False	True
9	False	False	True

```
Model Training Completed Successfully.
```

```
-----  
Decision Tree Prediction  
-----
```

```
New Sample:
```

```
Experience Performance Leadership Training  
0      High    Excellent      Yes      Yes
```

```
Predicted Result:
```

```
Yes
```

```
Decision Tree Rules:
```

```
|--- Leadership_Yes <= 0.50  
|   |--- Performance_Good <= 0.50  
|   |   |--- class: No  
|   |   |--- Performance_Good > 0.50  
|   |   |--- class: Yes  
|--- Leadership_Yes > 0.50  
|   |--- class: Yes
```

Result:

The ID3 Decision Tree algorithm was successfully implemented using the Entropy criterion. The trained models classified all five new samples successfully.